

Cyber-Attack Attribution using Malware Artifacts

The University of Texas at Dallas
December 4, 2020
Team Ctrl-Alt-Elite

Kavya Kurup
Sydney Medina
Dhruv Mehta
Juhi Patel
Pritom Das Radheshyam

kavya.kurup@utdallas.edu
sydney.medina1@utdallas.edu
dhruvhemant.mehta@utdallas.edu
juhi.patel1@utdallas.edu
pritomdas.radheshyam@utdallas.edu

Table of Contents

Table of Contents	2
Project Summary	3
Description	3
Motivation	3
Organization	3
Required Skills	3
Total Budget	4
Team Approach and Justification	4
Technical Approach	4
Data Gathering	4
Data Classification, Extraction, and Use	5
Data Preprocessing	6
Machine Learning Model	7
Future Work	9
Literature Review	9
DeepAPT: Nation-State APT Attribution Using End-to-End Deep Neural Networks	9
MV FCC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution	9
Applying Supervised Learning on Malware Authorship Attribution	10
Deep learning based Sequential model for malware analysis using Windows	10
The Power of Threat Attribution- Kaspersky Threat Attribution Engine	11
When Coding Style Survives Compilation: De-anonymizing Programmers from Executable Binaries	12
Biographical Sketches of Investigators	13
Kavya Kurup	13
Sydney Medina	13
Dhruv Mehta	13
Juhi Patel	13
Pritom Das Radheshyam	14
Bibliography	15

Project Summary

Description

Cyber attack attribution is the process of attempting to trace back a piece of code or malware to a perpetrator of a cyberattack. As cyber attacks have become more prevalent, cyber attack attribution is becoming more valuable. The process of cyber attack attribution can be done using reverse engineering. From the metadata of the malware executable file, we can gather data such as date of creation, variable names used, and what library calls are imported. This information can be used as features for attribution analysis. We need to extract the features from malware that can be used for attribution and analyse them using some technique to attribute the attacks.

Motivation

Our interest stems from the increase in recent cyber attacks. It is important to analyse the malware and get its artifacts which can give us a hint of the attacker. Cyber attacks will only become more prominent as time goes on and as our society becomes more dependent on our technological infrastructure. Cyber attack attribution will be a key element in combating malware development by helping to classify the malware by type, authorship, and other key categories. This is especially useful when you consider state-sponsored malware. It is becoming imperative to be able to identify the authorship of the malware as well as the country the malware comes from because cyber warfare will eclipse traditional combat warfare in the future. Our team has experience with machine learning algorithms and is proficient with Python.

Organization

Required Skills

- A basic understanding of malware analysis and machine learning algorithms.
- Knowledge of how VirusTotal works.
- Knowledge of different artifacts of malware such as PE header.
- Effective communication with our teammates and the sponsor.

Total Budget

There are no costs related to this project. The team will spend approximately 10 hours a week working on this project.

Team Approach and Justification

We plan to use machine learning to attribute a nation-state sponsored malware to an APT group. The malware samples will be retrieved from GitHub repositories we find during research. We will generate reports of these malwares using the VirusTotal API. We will extract various attributes from the VirusTotal reports and use them as an input to our machine learning agent to get the attribution result, which then be utilized to classify the malware into different APT groups. The justification we have for this approach comes from the research paper “Applying Supervised Learning on Malware Authorship Attribution” and other papers noted in our included Lit review. The authors of these papers classify state sponsored malware into countries and API groups by using the reports generated by various sandbox environments.

Technical Approach

First, our team identified a GitHub repository of pre-classified state sponsored malware that we will be using as our dataset. We then performed static analysis on the malware samples. We decided not to proceed with dynamic analysis since the setup of Cuckoo would take a lot of time and the accuracy of static analysis was considered very good. We did not use any specific environment for malware analysis, since we did not perform dynamic analysis and there was no need to execute the malware samples. We have generated the VirusTotal reports for the malware samples. We have fetched various attributes from the VirusTotal reports and use those attributes as inputs to our Random Forest Algorithm

Data Gathering

As we mentioned in the initial project proposal, during our research we found a repository in Github (<https://github.com/cyber-research/APTMalware>) that contained about 4,500 country-sponsored malware samples [3] that we decided to use as our dataset. We also have access to the VirusTotal Developers API and have created a

script that will send the hash value of all malware samples to the VirusTotal API and download the reports for all malwares as a json file.

Data Classification, Extraction, and Use

Since we are doing static analysis, we needed to consider whether the malwares were packed or not. Packing is a form of code obfuscation that modifies the format of the malware by compressing or encrypting the data, so we would not be able to extract the artifacts we need without putting the malware through some type of unpacking software. Because of this, we used a tool called PEiD that can detect whether or not a malware sample is packed and classified all of our malware into either unpacked or packed categories. The number of packed and unpacked malwares per APT group are summarized in the table below (Fig 1). We have only used unpacked malware samples for attribution to APT Group. The number of unpacked malware samples are 3591.

APT GROUP	No. of unpacked files	No Information	No of packed files
1	353	46	1
10	466	140	1
19	474	172	3
21	57	29	0
28	68	132	3
29	71	413	5
30	193	421	38
Dark_Hotel	330	552	39
Energetic_Bear	330	680	40
Equation_Group	721	684	43
Gorgon_group	886	759	147
Winnti	1086	861	152

Fig 1

We have extracted the following attributes from the VirusTotal reports for each malware sample

1. pe-resource-langs

2. imports
3. pe-entry-points

We created a python script that would extract the above attributes from each of the json malware report files and consolidate it into one csv file. We created a CSV file with the mentioned three attributes, resource (i.e hash of the malware) and APT group columns which we fetched from the GitHub repository. Below is a snapshot of the unprocessed dataset (Fig 2).

Resource	APTGroup	Entry Point	Language	Library
4d74c3de7274bEquation Gr		54299	['NEUTRAL', 'ENGLISH US']	['ADVAPI32.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'WS2_32.dll', 'USER32.dll']
cc223465dec38APT 21		7627	['ENGLISH US', 'CHINESE SIMPLIFIED']	['KERNEL32.dll']
a49713feddf87APT 1		38026	['NEUTRAL DEFAULT', 'ENGLISH US']	['GDI32.dll', 'ADVAPI32.dll', 'KERNEL32.dll', 'OLEAUT32.dll', 'SHELL32.dll', 'ole32.dll', 'SHLWAPI.dll', 'USER32.dll', 'COMCTL32.dll']
78c00614035b9Gorgon Gro		390884	['NEUTRAL', 'ENGLISH US']	['comctl32.dll', 'version.dll', 'gdi32.dll', 'kernel32.dll', 'oleaut32.dll', 'advapi32.dll', 'user32.dll', 'comctl32.dll']
44787cdd191c10APT 21		6948	['ENGLISH US']	['SHELL32.dll', 'KERNEL32.dll']
f43d83ef124b9Gorgon Group			['']	['']
2f1e000fae9b1APT 1		103376	['FRENCH', 'ENGLISH US']	['ADVAPI32.dll', 'PSAPI.dll', 'Secur32.dll', 'KERNEL32.dll']
0bceff0cd78ceAPT 1		35109	['ENGLISH US']	['WSVCP90.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'WININET.dll', 'USER32.dll']
7040b1e91d48aEquation Gr		54299	['NEUTRAL', 'ENGLISH US']	['ADVAPI32.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'WS2_32.dll', 'USER32.dll']
48b97355589cfDark Hotel		49648	['NEUTRAL']	['advapi32.dll', 'kernel32.dll', 'user32.dll']
208f054da3f64cAPT 30		28202	['CHINESE SIMPLIFIED']	['MPR.dll', 'SHELL32.dll', 'KERNEL32.dll', 'WSOCK32.dll', 'NETAPI32.dll', 'ADVAPI32.dll', 'ole32.dll', 'SHLWAPI.dll', 'USER32.dll']
5dd55e32b1c31APT 10		8009	['ENGLISH US', 'CHINESE SIMPLIFIED']	['ADVAPI32.dll', 'KERNEL32.dll', 'USER32.dll']
4b74c930c9d9ceAPT 28		89645	['ENGLISH US']	['gdiplus.dll', 'GDI32.dll', 'KERNEL32.dll', 'ADVAPI32.dll', 'ole32.dll', 'SHLWAPI.dll', 'PSAPI.dll', 'USER32.dll']
50cddcf907e2c2APT 1		14463	['CHINESE SIMPLIFIED']	['ADVAPI32.dll', 'SHELL32.dll', 'KERNEL32.dll', 'LZ32.dll', 'MSVCRT.dll']
d5c3b0c1d602cAPT 1		20759	['']	['WSVCP90.dll', 'WININET.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'NETAPI32.dll', 'ADVAPI32.dll', 'WS2_32.dll', 'USER32.dll']
009680740c9e31Dark Hotel		9664	['KOREAN']	['tchlpapi.dll', 'WININET.dll', 'SHELL32.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'ADVAPI32.dll', 'SHLWAPI.dll', 'WS2_32.dll', 'USER32.dll']
cd91cc59bdc38Equation Gr		54299	['NEUTRAL', 'ENGLISH US']	['ADVAPI32.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'WS2_32.dll', 'USER32.dll']
437c0c5a30b4cAPT 29		8204	['']	['']
ca3900d33c7ddAPT 1		12282	['CHINESE SIMPLIFIED']	['SHELL32.dll', 'KERNEL32.dll', 'MSVCRT.dll', 'WININET.dll', 'USER32.dll']
2a836894d3c5cAPT 19		238148	['ENGLISH US']	['fsccdd.sys', 'ntoskrnl.exe']

Fig 2: Unprocessed Data

Data Preprocessing

Next, the data needed to be cleaned in order for it to be inputted into the machine learning model. Brackets and commas in the language and library column were removed and all values were formatted to be uppercase so the model will not differentiate between them and give inaccurate results. Then, the column for the library and language features were one hot encoded. This resulted in creation of dummy variables for each unique library and language value. Then, the APT groups were mapped to integers so they can properly be used in the classifier model. Lastly, we removed 792 rows with null values in multiple columns. After preprocessing, we were left with 148 features and 2862 rows. Below is the snapshot of the preprocessed data (Fig 3).

Resource	APT Group	Entry Point	'ACTIVED'	'ADVAPI3'	'API-MS-W'	'API-MS-V'	'API-MS-V'	'API-MS-V'	'API-MS-V'	'API-MS-V'	'ATL71.DL'	'ATL80.DL'	'AVICAP3'	'AVIFIL32'	'CABINET'	'COMCTL3'	'COMDLG'	'COMSVC'	'CRT.DLL'	'CRYPT32'	'D3D9.DL'	'D3DX9_3'	'D3DX9_4'
4d74c8da	Equation Gr	54299	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
cc221465	APT 21	7627	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
a49718fe	APT 1	38026	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
78c00614	Gorgon Grov	396884	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
44787ddf	APT 21	6948	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2f1e006f	APT 1	103376	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0bfceffb5	APT 1	35109	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7640b1a9	Equation Gr	54299	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
49b97355	Dark Hotel	45648	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
263f094d	APT 30	28202	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9ddd5e3	APT 10	8069	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4b74c90c	APT 28	83645	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
50ddcf95	APT 1	14463	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
d5eabcd	APT 1	20759	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
60968074	Dark Hotel	9664	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
ed61da9f	Equation Gr	54299	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
43fa0d5a	APT 29	8204	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
ca3960d3	APT 1	12282	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2e836934	APT 19	258148	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0aa3a3e	APT 21	7156	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
d3632c57	APT 10	63945	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
273bb41e	Energetic Be	129578	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
d0db619e	APT 28	113828	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1cb18260	APT 19	6477	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
1c90ecf95	Energetic Be	95398	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
1ce04952	APT 29	4157	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
a1e31786	APT 1	14511	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fig 3: Preprocessed Data

Machine Learning Model

For our machine learning analysis, we decided to use Random Forest since it uses multiple decision trees which prevent overfitting. Also, it is very easy to calculate the important features in Random Forest. As well as, the Random Forest protects error correlation between prediction trees. We chose a 70-30 split for the data for training and testing. We used the scikit-learn RandomForestClassifier function. The initial value for hyperparameters of the RandomForestClassifier were:

- min_samples_leaf=50,
- n_estimators=150,
- bootstrap=True,
- oob_score=True,
- n_jobs=-1,
- random_state=seed,
- max_features='auto'

The model initially achieved 58% accuracy. To increase the accuracy, we did some hyperparameter tuning. We adjusted the number of estimators to 300 and the minimum

number of leaf samples to 3 and achieved an accuracy of 83%. Further hypertuning did not achieve better results, so we called it at 83%.

To ensure our model is not overfitting or underfitting, we performed cross-validation. For 20 fold random cross-validation, we achieve an accuracy of 86% which proved the model was not underfitting or overfitting

We also calculated the features that contributed most to the classification.(Fig 4). The attribute entry point is the most important attribute followed by neutral which is the value for the pe-resource-langs.

	importance
Entry Point	0.1190793549
NEUTRAL*	0.1042488131
MSVCRT.DLL*	0.06959561717
WS2_32.DLL*	0.05863191299
WININET.DLL*	0.05771601752
USER32.DLL*	0.04198525689
ENGLISH US*	0.03396263127
OLE32.DLL*	0.03341085389
SHLWAPI.DLL*	0.03245535002
SHELL32.DLL*	0.03183228205
CRYPT32.DLL*	0.03098899175
CHINESE SIMPLIFIE	0.02996419383
KERNEL32.DLL*	0.02674821228
VERSION.DLL*	0.02405199711
ADVAPI32.DLL*	0.02381425892
COMCTL32.DLL*	0.02339712023
MSCOREE.DLL*	0.02296011856
GDI32.DLL*	0.02221921503
OLEAUT32.DLL*	0.02146662713
MPR.DLL*	0.02008456941
IPHLPAPI.DLL*	0.01715311222
URLMON.DLL*	0.0156822925
WSOCK32.DLL*	0.01440986185
COMDLG32.DLL*	0.01000456097
MFC42.DLL*	0.009786070476
SECUR32.DLL*	0.008958750082
KOREAN*	0.007736578064

Fig 4: Important Features

Future Work

The model was quite efficient with only three main features and 2862 malware samples. The accuracy of the model may increase by using more features such as entropy, no of sections or more malware samples. We extracted less features due to time constraint and there is a lack of APT malware datasets. As well as, the data preprocessing was very rigorous and took a lot of time.

In the future, it would be good for someone to extract more attributes from the VirusTotal report and incorporate it into the dataset. As well as, perform dynamic analysis using Cuckoo sandbox and use neural networks for classification.

Literature Review

DeepAPT: Nation-State APT Attribution Using End-to-End Deep Neural Networks^[6]

The authors here describe how the nation-state APT can be attributed by using its dynamic analysis as raw input to a deep neural network (DNN) allowing it to learn high level feature abstractions of the APTs itself. The nation-state APT problem is more complex since a nation may have many cyber units with different targets, functionality and framework. Therefore, the author attribution approach will not work. APT's cannot be classified based only on frameworks and functionality. The authors used the Cuckoo Sandbox reports of samples of APT which were allegedly developed by nation-states as input to DNN. The DNN had a 10 layer fully connected network. The classifier achieved an accuracy of 94.6%. The authors concluded different nation-states use different APT developing methodologies, such that the works of developers in separate cyber units are still sufficiently similar to each other.

MVFCC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution^[4]

The authors proposed a multi-view fuzzy consensus clustering model for attributing malware. The authors used the Cuckoo sandbox reports for the malware samples and converted the report for each malware to different views in four categories namely Opcode, Bytecode, SystemCall and Header. The opcode and bytecode consists of the Binary, count, Frequency of occurrence, Term Frequency-Inverse Document Frequency and Eigen Vector modes. The authors then fuzzified each feature of all views using Trapezoidal and Triangular fuzzifier. They next used the Fuzzy Pattern tree (FPT)

classification algorithm on views. FPT will result in a set of k (k = no of APT groups) classifiers being trained on each view. The authors now applied Fuzzy C-Means partitioning technique to divide malware in each APT group to several clusters. Classification and clustering provided a database of fuzzy metrics on which a decision tree was trained to provide the APT group for the malware sample. The decision tree aggregates information from different fuzzy learning algorithms. The authors were able to attribute malware samples to its APT actors correctly with 95% of accuracy. Therefore, a malicious actor needs to change all possible properties of the malware to evade or fool the MVFCC

Applying Supervised Learning on Malware Authorship Attribution [1]

The authors of these papers proposed a method to attribute state-sponsored authorship of malware using the supervised learning approaches Ro18 and Am17. Ro18 is a supervised learning technique that analyzes malware reports generated by Cuckoo Sandbox and inputs it into a deep neural network in order to classify state-sponsored malware. Am17 is similar to Ro18, but focuses on family classification of criminal malware instead of just state-sponsored malware, and it uses a random forest classifier instead of a DNN. The results from Ro18 when given a scenario of unseen APT groups showed that the accuracy of the classifiers was highly dependent on the sandbox reports that were used. On the other hand, when working with samples of earlier seen APT groups, the DNN works much better overall, regardless of the sandbox environment used. When using Am17, the results with unseen APT groups are worse than the DNN solution, while working with seen APT groups yields similarly good results. <https://github.com/cyber-research/APTMalware>

Deep learning based Sequential model for malware analysis using Windows exe API Calls^[3]

The authors experimented with classifying malware using a sequential model machine learning technique in a Cuckoo sandbox environment. The Long-Short Term model learning method was chosen because it allows long term dependency to be detected at random intervals. Using multiple malware datasets, they were able to test the LSTM model, but also develop data on API calls for Windows OS malware. To do so, they first started with filtering out rows that did not have 10 or more API calls. Next, they compared the hash values given by VirusTotal with the hash values of each of the malware samples to try and classify its type. Lastly, they used the joint data of the API

call sequence and the malware classification to create and train a dataset. For this, decision trees, k-clustering, and support vector machines methods were used. The authors were able to test and classify malware behavior for 7107 Windows software samples. The success rate was between 83.5% and 98.5% with performance not being an issue. Although this experiment was more of a proof of concept, it did provide more insight of Windows API calls and malware correlation, as well as a dataset pertaining to this work.

The Power of Threat Attribution- Kaspersky Threat Attribution Engine^[5]

Tracking, analyzing, interpreting and mitigating constantly evolving IT security threats is a massive undertaking. Threat intelligence (TI) has true value beyond the current hype around an emerging pocket in the information security industry. TI includes a number of different services that help organizations stay protected, increase awareness and assist with investigations. When it comes to threat intelligence, threat attribution is probably the most visible process and also a point of contention.

And there is a definite reason for this. The average time from detection to response against highly sophisticated threats is usually too long due to complex investigation and reverse engineering processes. Today, in the digital era, organizations are obliged to instantly investigate, prioritize all alerts and accelerate the time to respond. Correct and timely attribution helps not only to shorten incident response times from hours to minutes but also to reduce the number of false positives. Attributing targeted attacks is a powerful and essential tool that: Evaluates if you are a target or an unintended victim, Provides insights into the actors behind the attack and their motivation, Enables effective detection, investigation, containment and response based on the knowledge of the tactics, techniques and procedures specific to the threat actor.

Some major threat attribution techniques are (1)Timestamps:One major benefit of the Portable Execution (PE) format is that compilation times are always included. Though they can be easily altered, many samples have the original timestamps. There are a few important indicators that timestamps can show. (2)Strings, debug paths and metadata: Implanted binary code can also expose interesting details about the authors. The artifacts include their preferred language as well as debug paths, which often reveal a username, as well as internal naming conventions for internal tools, projects or campaigns. Also metadata that might reveal the configuration of the developer's system. (3)Infrastructure and backend: infrastructure reuse is the most telling sign that

an attacker is resurfacing or retooling. When attackers retrieve data from the email account of an exfiltration server, prepare staging or phishing servers or check on the availability of a compromised domain, they almost always use an anonymizing service. (4)Exploits: Zero-day exploits are a great source of details about the actor behind an attack. The presence of a “0-day” immediately sets an actor apart from ordinary attackers. (5)Targeted victim: Though many indicators may be faked or altered, the dynamic between attacker and victim is harder to hide or directly manipulate as it involves ‘real-world’ publicly known circumstances or geopolitical conflicts. This insight allows for attacker profiling. A possible outcome is the mapping of a campaign to a geopolitical or regional situation that may point in the direction of a given perpetrating organization or nation.

When Coding Style Survives Compilation: De-anonymizing Programmers from Executable Binaries^[2]

There is a way to identify authors of a computer program based on their coding style: variable names, spacing, etc... It has been shown that source code can be attributed to authors with high accuracy, although attribution of executable binaries seem to be a lot more difficult. They used machine learning to find patterns and they decompiled the executable binary to source code. They evaluated their approach from data from Google Code Jam and had an attribution accuracy of up to 96%.

This is preserved in the compilation process and this can be extracted from the executable binary. It may be possible to infer the programmers identity if we have a set of KNOWN potential candidate programmers, along with executable binary samples known to be authored by these candidates. This has a forensics application as they can consistently identify virtual personas and individual malicious cyber operators over time. We see hackers try to obscure their binaries so it cannot be traced back to them although we find that traditional binary obfuscation, enabling compiler optimizations, or stripping debugging symbols in executable binaries results in only a modest decrease in de-anonymization accuracy. We emphasize that research challenges remain before programmer de-anonymization from executable binaries is fully ready for practical use. For example, programs may be authored by multiple programmers and may have gone through encryption.

They approach this problem using supervised machine learning, that is, we generate a classifier from training data of sample executable binaries with known authors. A drawback is that the method is inoperable if samples are not available or too short to represent authorial style.

Biographical Sketches of Investigators

Kavya Kurup

Kavya Kurup holds a Bachelor's degree in Software Engineering and is currently pursuing her Master's degree in Computer Science with a concentration in Information Assurance. Also, she is currently a systems engineer at T-Mobile. Through her schooling and work, Kavya has developed applications in Python and has experience with network and data analysis. Her interests lie in vulnerability research, cryptography, and network security.

Sydney Medina

Sydney Medina is in her final semester of her undergraduate degree in computer science and is currently enrolled in fast track to pursue a masters in Information Assurance. She is currently working a Cyber Security internship at Cisco working on the Incident Response and Digital Forensics team. Last summer she interned at Lockheed Martin on the Cyber Security Blue Team. Working on these teams she has gained knowledge and familiarity with cyber security tools, and jargon, as well as knowledge of how popular cyber attacks work since she sees them being used on a daily basis during her internship.

Dhruv Mehta

Dhruv Mehta holds a Bachelor's degree in Information Technology from India and is currently pursuing his Master's degree in Computer Science with a concentration in Information Assurance. He is a Certified Ethical Hacker (Master). Being a CEH, he has the knowledge and experience of penetration testing and its common tools. He and his team were able to find three critical vulnerabilities in a Wi-Fi Garage door camera. He is proficient in Java and Python. He has taken coursework in Machine Learning, Network Security, Information Security. Currently, he has taken the course Systems Security and Malicious Code Analysis. His interests are penetration testing, vulnerability assessment and network security.

Juhi Patel

Juhi Patel is in her final semester of her undergraduate degree in Computer Science and is currently enrolled in fast track to pursue a masters in Information Assurance. This

past summer she has been working on designing and developing an app at UTSW that focuses on gathering data from young adults to see if they are at risk/already have depression. She has taken coursework for machine learning, database design, algorithm design, and operating systems. Currently, she is also taking a course in systems security and malicious code. She has a strong background with Java and C++ with some experience in MIPS and C.

Pritom Das Radheshyam

Pritom Das Radheshyam is currently pursuing a Master's degree in Computer Science and has earlier graduated with a Bachelor's degree in Information Technology. He is well versed in Machine Learning, Cloud Computing and Big Data Technologies. He is an IBM certified Cloud Computing and Virtualization Expert. He has a strong background with Python and Data Visualization. His interests lie in cloud security, network security and vulnerability research.

Bibliography

[1] Boot, Coen. *Applying Supervised Learning on Malware Authorship Attribution*. Diss. Radboud University Nijmegen, 2019.

[2] Aylin Caliskan, Fabian Yamaguchi, Edwin Dauber, Richard Harang, Konrad Rieck, Rachel Greenstadt, Arvind Narayanan “When Coding Style Survives Compilation: De-Anonymizing Programmers from Executable Binaries.” N.p., 2015. Web.
<https://arxiv.org/abs/1512.08546>

[3] Ferhat Ozgur Catak, Ahmet Faruk Yazı, Ogerta Elezaj, Javed Ahmed “Deep Learning Based Sequential Model for Malware Analysis Using Windows Exe API Calls.” PeerJ. Computer science 6 (2020): e285–. Web.
<https://peerj.com/articles/cs-285/>

[4] Haddadpajouh, Hamed ; Azmoodeh, Amin ; Dehghantanha, Ali ; Parizi, Reza M. “MV FCC: A Multi-View Fuzzy Consensus Clustering Model for Malware Threat Attribution.” IEEE access 8 (2020): 139188–139198. Web.

[5] Kaspersky, Eugene, et al. “The Power of Threat Attribution-Kaspersky Threat Attribution Engine”
<https://media.kaspersky.com/en/business-security/enterprise/threat-attribution-engine-whitepaper.pdf>

[6] Rosenberg, Ishai ; Sicard, Guillaume ; David, Eli “DeepAPT: Nation-State APT Attribution Using End-to-End Deep Neural Networks.” *Artificial Neural Networks and Machine Learning – ICANN 2017 Lecture Notes in Computer Science*, 2017, pp. 91–99., doi:10.1007/978-3-319-68612-7_11.